

# C# Statistics Code

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        double[] data = {
            115, 182, 191, 31, 196, 1099, 5, 172, 10, 179,
            83, 21, 20, 21, 186, 177, 195, 193, 188, 199,
            62, 109, 105, 183, 110
        };

        int n = data.Length;

        var sorted = data.OrderBy(x => x).ToArray();

        double mean = data.Average();

        double median = (n % 2 == 0) ?
            (sorted[n / 2 - 1] + sorted[n / 2]) / 2 :
            sorted[n / 2];

        var mode = data.GroupBy(x => x)
            .OrderByDescending(g => g.Count())
            .First().Key;

        double variance = data.Sum(x => Math.Pow(x - mean, 2)) / n;

        double stdDev = Math.Sqrt(variance);

        double P20 = Percentile(sorted, 20);
        double P50 = Percentile(sorted, 50);

        double Q1 = Percentile(sorted, 25);
        double Q2 = median;
        double Q3 = Percentile(sorted, 75);

        double range = sorted.Max() - sorted.Min();

        double IQR = Q3 - Q1;

        double sum = data.Sum();

        Console.WriteLine("Mean: " + mean);
        Console.WriteLine("Mode: " + mode);
        Console.WriteLine("Median: " + median);
        Console.WriteLine("Variance: " + variance);
        Console.WriteLine("P20: " + P20);
        Console.WriteLine("P50: " + P50);
        Console.WriteLine("Q1: " + Q1);
        Console.WriteLine("Q2: " + Q2);
        Console.WriteLine("Q3: " + Q3);
        Console.WriteLine("Range: " + range);
        Console.WriteLine("IQR: " + IQR);
        Console.WriteLine("Standard Deviation: " + stdDev);
        Console.WriteLine("Summation: " + sum);
    }

    static double Percentile(double[] sortedData, double percentile)
    {
        double position = (percentile / 100.0) * (sortedData.Length + 1);
        int index = (int)position;

        if (index <= 0) return sortedData[0];
        if (index >= sortedData.Length) return sortedData[sortedData.Length - 1];

        double fraction = position - index;
        return sortedData[index - 1] + fraction * (sortedData[index] - sortedData[index - 1]);
    }
}
```